Step 2b — Dataset: generare dati alla MiniCheck e alla FactCG

Obiettivo

Nel piano generale la generazione era un'aggiunta. Qui è il **centro**: i due paper mostrano che, a parità di backbone, sono i dati sintetici a fare la differenza (MiniCheck-FT5 75,0; FactCG-DBT 75,6; con lo stesso DeBERTa e dati normali AlignScore si ferma a 70,5). Questo step descrive come produrre quattro famiglie di dati:

1. **C2D** (claim → documento) e **D2C** (documento → claim), come MiniCheck;
2. **CG2C-MHQA** e **CG2C-Doc** (grafo → claim multi-hop), come FactCG;
3. **negativi per modifica minima** (`remove_evidence`, `contradict_claim`, già prototipati in `llm_gen/`);
4. **verifica delle etichette**.

Regole invariate rispetto a ../fine_tuning/02b_dataset_generazione_llm.md: si genera **solo dal train**; **l'etichetta deriva dall'operazione**, non dal giudizio dell'LLM; un secondo LLM giudice filtra.

1. MiniCheck: usare i dati rilasciati, poi estenderli

I 14k esempi C2D/D2C sono pubblici (`lytang/C2D-and-D2C-MiniCheck`, MIT) e già in `data/finetune/raw/minichec`
Si usano interi. Estenderli serve perché 14k è poco per un 12B e perché i documenti C2D sono tutti scritti da GPT-4 nello stesso stile.

1.1 C2D esteso

Procedura (dettagli e formule in ../models/minicheck.md, §2):

Passo	Chi lo fa	Esempio
Claim di partenza	claim reali dal train di FEVER, HoVer, AVeriTeC (non da AggreFact)	"The Tijuana stadium, with a capacity of 3,000, hosted over 5,000 caravan members."
Decomposizione in fatti atomici `a_1 … a_l`	LLM	a1: oltre 5.000 membri nello stadio; a2: capacità 3.000
Per ogni fatto, **due frasi** che solo insieme lo implicano	LLM	"La ristrutturazione ha raddoppiato la capacità." + "Prima poteva ospitare 1.500 persone."
Documento con tutte le frasi	LLM	`D = PassageGen(s)` → positivo `(D, c, 1)`
Documento senza una delle frasi	LLM	`D' = PassageGen(s \ s_2,2)` → negativo `(D', c, 0)`
Power set dei fatti	LLM (fusione dei sotto-claim)	`(D', "lo stadio ha capacità 3.000", 0)`, `(D', "oltre 5.000 membri nello stadio", 1)`

Il power set è ciò che rende C2D efficiente: un documento bucato produce sia negativi sia positivi, quindi il modello non può imparare "documento sintetico corto = falso".

1.2 D2C esteso

Passo	Dettaglio
Documenti reali	notizie e trascrizioni **non** presenti in AggreFact: per esempio il train di CNN/DM e XSum **escludendo gli articoli dei test** di AggreFact (controllo a 13-grammi), trascrizioni QMSum/AMI, documenti di SummEdits train
Chunk e riassunto	3 chunk per documento; un LLM riassume ciascuno → claim `c_i`, positivo `(D_i, c_i, 1)`
Ablazione	si toglie una frase alla volta da `D_i`; un giudice dice quali fatti di `c_i` restano supportati → positivi e negativi
Cross-chunk	gli stessi sotto-claim contro gli altri chunk dello stesso documento

D2C è il dato più vicino alle fonti deboli dei nostri modelli (CNN 62–72, MediaSum 67–73): riassunti di notizie e trascrizioni. È la priorità della corsa piena.

Attenzione alla contaminazione: CNN/DM, XSum, MediaSum e MeetingBank sono fonti di AggreFact. Si possono usare **solo** documenti che non compaiono in AggreFact (dev e test) secondo il controllo a 13-grammi di `make_mix.py`, e conviene preferire corpora diversi con lo stesso stile (per esempio altre raccolte di notizie o trascrizioni).

2. FactCG: rigenerare CG2C

I dati di FactCG non sono rilasciati; il codice sì (derenlei/FactCG, MIT). Due varianti (../models/FactCG.md, §2):

2.1 CG2C-MHQA (da HotpotQA / MuSiQue)

Passo	Esempio
Domanda + risposta → claim dichiarativo	Q: “In which country is the moth genus whose only species is I. pectinicornalis found?” A: “India” → “The moth genus whose only species is I. pectinicornalis is found in India.”
Grafo dalle frasi di supporto, sottografo del claim	Indogrammodes → I. pectinicornalis → India
Negativo: un LLM toglie **una** relazione del sottografo dal documento	si riscrive la frase “… which is found in India” senza il paese
Filtro NLI	si scartano le coppie che un modello NLI (RoBERTa-MNLI) indovina già

Il filtro NLI di FactCG tiene solo i negativi **difficili**. Da noi si può usare come filtro il nostro modello di partenza (KnowLine-4B, una domanda), come per ANLI difficile: si tengono le coppie su cui sbaglia o è incerto.

2.2 CG2C-Doc (da documenti in prosa)

Grafo di contesto dal documento → sottografi **lineari** (senza rami: FactCG mostra che rendono un po’ di più) di 2–4 archi → un LLM scrive un claim che attraversa tutti i nodi (positivo) → si toglie dal documento la relazione di un arco scelto a caso (negativo). Documenti di partenza: pagine HoVer, FEVER, documenti D2C.

3. Negativi per modifica minima (prototipo esistente)

Già in `llm_gen/` (task `remove_evidence`, `contradict_claim`, `verify_label`): restano utili come terza famiglia, soprattutto `remove_evidence` su documenti **lunghi** (SummEdits, D2C), che insegna a dire "no" quando l'informazione manca in un testo di migliaia di token.

4. Verifica

Ogni esempio generato passa da un giudice (modello diverso dal generatore) con risposta *supported / contradicted / not_enough_info*:

Famiglia	Si tiene se il giudice dice
C2D/D2C positivi	supported
C2D/D2C negativi per frase tolta, `remove_evidence`, CG2C negativi	not_enough_info (se dice contradicted, la cancellazione ha creato una contraddizione: si scarta o si rietichetta come "contraddetto")
`contradict_claim`	contradicted
CG2C positivi	supported

Tasso di scarto atteso 10–30%; sopra 40% il prompt va rivisto.

5. Quale LLM

Come nel piano generale (../fine_tuning/02b_dataset_generazione_llm.md, §6): generazione in volume con un modello aperto in locale (Qwen3-30B-A3B, gpt-oss-20b su L40S), giudice diverso e più forte (gpt-oss-120b su 2× L40S, Gemma-4-31B). Con modelli aperti non ci sono vincoli d'uso degli output; con API commerciali vanno controllati i termini.

6. Volumi

Famiglia	Pilota	Corsa piena	Note
MiniCheck rilasciato (C2D + D2C)	14k	14k	sempre
C2D esteso	0	10–20k	dopo il pilota
D2C esteso (notizie, trascrizioni)	3k	15–30k	priorità: è il dominio debole
CG2C-MHQA	0	10–20k	esperimento D-CG
CG2C-Doc	0	5–10k	esperimento D-CG
`remove_evidence` su documenti lunghi	2k	5–10k	
`contradict_claim`	2k	5k	

Costo: C2D costa ~5–8 chiamate per claim (decomposizione, coppie di frasi, documenti, fusioni del power set) ma produce 6–10 esempi; D2C ~4 chiamate per chunk. 20k esempi verificati ≈ 50–80k chiamate ≈ 100–150M token: 1–2 giorni con un modello locale su L40S.

7. Codice da scrivere

Nuovi task in `llm_gen/tasks.py`: `c2d_decompose`, `c2d_sentpair`, `c2d_passage`, `c2d_merge`, `d2c_summarize`, `d2c_entail`, `cg2c_graph`, `cg2c_claim`, `cg2c_remove_relation`. Per CG2C conviene partire dai prompt del repository di FactCG (tabelle 13–16 del paper) e adattarli al client di `llm_gen/`.

Criteri per passare allo step successivo

- Per ogni famiglia: tasso di accettazione su ≥ 200 esempi; 30 esempi accettati letti a mano con < 3 etichette sbagliate.
- Decontaminazione dei documenti D2C contro AggreFact registrata.
- File verificati in `data/finetune/raw/<famiglia>.jsonl` con `doc_id` ereditato.